(1) Signals$_{tx}$ = $\{s(t); 0; -s(t)\}$

$Z(t)$ – complex WGN

$E[Z(t)] = 0$

$R_z = E[Z(t) Z^*(\tau)] = 2N_0 \delta(t-\tau)$

Signal$_{rx}$ = $\tilde{r}_t(t) = \begin{cases} s(t) + Z(t) \\ -s(t) + Z(t) \\ Z(t) \end{cases}$

$U = \mathrm{Re}\left\{ \int_0^T \tilde{r}_t(t) s^*(t) dt \right\} \rightarrow \begin{cases} U > A \Rightarrow s(t) \\ -A < U < A \Rightarrow 0 \\ U < -A \Rightarrow -s(t) \end{cases}$

$P(\text{transmit } s(t)) = P(\text{transmit } 0) = P(\text{transmit } -s(t)) = \frac{1}{3}$

---

1) $\int_0^T Z(t) s^*(t) dt = Z$ ← It's complex Gaussian random variable, because we take a linear functional of a Gaussian random process.

- $E[Z] = \int_0^T E[Z(t)] s^*(t) dt = 0$

- $E[Z Z^*] = D[Z] = \int_0^T \int_0^T \underbrace{E[Z(t) Z^*(\tilde{t})]}_{2N_0 \delta(t-\tilde{t})} s^*(t) s(\tilde{t}) dt d\tilde{t} = 2N_0 \int_0^T |s(t)|^2 dt$

$= 2N_0 E_s$, where $E_s = \int_0^T |s(t)|^2 dt$.

$\Leftarrow$ $E[Z \cdot Z] = \int_0^T \int_0^T E[Z(t) Z(\tilde{t})] s^*(t) s^*(\tilde{t}) dt d\tilde{t} = 0$

$E[Z(t) Z(\tilde{t})] = E[a^2(t)] + E[i a(t) b(\tilde{t})] + (-1) E[b^2(\tilde{t})] = 0$

$Z(t) = a(t) + i b(t)$; $D[a(t)] = D[b(t)]$ $\Rightarrow$

Z – comp. WGN $\Rightarrow$ $E[a(t) b(\tilde{t})] = 0$

$\Rightarrow$ Z – "proper" variable $\Rightarrow$ $D[\mathrm{Im}(Z)] = D[\mathrm{Re}(Z)] = \frac{D[Z]}{2}$ $\Rightarrow$

$\Rightarrow D[\mathrm{Re}(Z)] = N_0 E_s$

$U = \mathrm{Re}\{Z\}$ ← Gaussian random variable

~~$U \sim N(0; \ldots)$~~; $U \sim N(0; N_0 E_s)$

2) $P_{err/s(t)} = P(U < A / s(t))$, $U = E_s + N(0; N_0 E_s) \Rightarrow$

$$P_{err/s(t)} = \int_{-\infty}^{A} \frac{1}{\sqrt{2\pi N_0 E_s}} \cdot \exp\left(-\frac{(x-E_s)^2}{2\sqrt{N_0 E_s}}\right) dx = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{A} \exp\left(\frac{-(x-E_s)^2}{\sqrt{N_0 E_s}\cdot 2}\right) \cdot d\left(\frac{x-E_s}{\sqrt{N_0 E_s}}\right) = \frac{1}{\sqrt{2\pi}} \int_{\frac{E_s-A}{\sqrt{N_0 E_s}}}^{\infty} \exp\left(\frac{-y^2}{2}\right) dy = Q\left(\frac{E_s-A}{\sqrt{N_0 E_s}}\right)$$

$y = \frac{E_s - x}{\sqrt{N_0 E_s}}$

$P_{err/-s(t)} = P(U > -A / -s(t))$, $U = -E_s + N(0, N_0 E_s) \Rightarrow$

$$P_{err/-s(t)} = \int_{-A}^{+\infty} \frac{1}{\sqrt{2\pi N_0 E_s}} \cdot \exp\left(\frac{-(x+E_s)^2}{\sqrt{2}\cdot\sqrt{2 N_0 E_s}}\right) dx = \frac{1}{\sqrt{2\pi}} \int_{\frac{E_s-A}{\sqrt{N_0 E_s}}}^{\infty} \exp\left(-\frac{y^2}{2}\right) dy = Q\left(\frac{E_s-A}{\sqrt{N_0 E_s}}\right)$$

$P_{err/0} = P(U < -A \cup U > A / 0)$, $U = N(0; N_0 E_s) \Rightarrow$

$$P_{err/0} = \int_{-\infty}^{-A} \frac{1}{\sqrt{2\pi N_0 E_s}} \cdot \exp\left(-\frac{x^2}{2\sqrt{N_0 E_s}}\right) dx + \int_{A}^{+\infty} \frac{1}{\sqrt{2\pi N_0 E_s}} \cdot \exp\left(\frac{-x^2}{2\cdot\sqrt{N_0 E_s}}\right) dx = \frac{1}{\sqrt{2\pi}} \int_{\frac{A}{\sqrt{N_0 E_s}}}^{+\infty} \exp\left(\frac{-y^2}{2}\right) dy + \frac{1}{\sqrt{2\pi}} \int_{\frac{A}{\sqrt{N_0 E_s}}}^{+\infty} \exp\left(-\frac{y^2}{2}\right) dy = 2\,Q\left(\frac{A}{\sqrt{N_0 E_s}}\right)$$

---

3) $P_{err} = P_{err/s(t)} \cdot P(s(t)) + P_{err/-s(t)} \cdot P(-s(t)) + P_{err/0} \cdot P(0) =$

$$= \frac{1}{3}\left(2 \cdot Q\left(\frac{E_s-A}{\sqrt{N_0 E_s}}\right) + 2\,Q\left(\frac{A}{\sqrt{N_0 E_s}}\right)\right) = \frac{2}{3}\left(Q\left(\frac{E_s-A}{\sqrt{N_0 E_s}}\right) + Q\left(\frac{A}{\sqrt{N_0 E_s}}\right)\right)$$

---

4) $A_{min} = \underset{A \in \mathbb{R}}{\operatorname{argmin}}\left(P_{err}\right)$

$$\frac{dP_{err}}{dA} = \frac{2}{3}\cdot\frac{1}{\sqrt{N_0 E_s}}\left((-1)\,q\left(\frac{E_s-A}{\sqrt{N_0 E_s}}\right)+q\left(\frac{A}{\sqrt{N_0 E_s}}\right)\right),\quad q=\frac{1}{\sqrt{2\pi}}\exp\left(\frac{-x^2}{2}\right)$$

$$\frac{dP_{err}}{dA}=0 \Rightarrow q\left(\frac{E_s-A}{\sqrt{N_0 E_s}}\right)=q\left(\frac{A}{\sqrt{N_0 E_s}}\right)\Rightarrow E_s-A=A \Rightarrow \boxed{A=\frac{E_s}{2}}$$

---

(2) $\vec{r}=\vec{s}+\vec{n}$, $\vec{n}=(n_1,n_2)$, $n_i \sim f(x)=\frac{1}{2}e^{-|x|}$

$s_1=(1,1)$
$s_2=(-1,-1)$; $E[n_1 n_2]=E[n_1]\,E[n_2]$

- $g(n_1,n_2)=f_1(n_1)\cdot f_2(n_2)=\frac{1}{4}e^{-(|x_1|+|x_2|)}$ ← $E[n_1 n_2]=E[n_1]E[n_2]$

$\vec{r}-\vec{s}=\vec{n} \Rightarrow g_1=\frac{1}{4}\exp\left(-\left(|x_1-1|+|x_2-1|\right)\right)=p(\vec{r}\,|\,\vec{s}_1)$

$f_x(x)\,dx=f_y(y)\,dy \Rightarrow$

$g_2=\frac{1}{4}\exp\left(-\left(|x_1+1|+|x_2+1|\right)\right)=p(\vec{r}\,|\,\vec{s}_2)$

- $D_1=\left\{\vec{r}\in\mathbb{R}^2 : p(\vec{r}\,|\,\vec{s}_1)>p(\vec{r}\,|\,s_2)\right\}$

$\frac{1}{4}\exp\left(-\left(|x_1-1|+|x_2-1|\right)\right)>\frac{1}{4}\exp\left(-\left(|x_1+1|+|x_2+1|\right)\right)\Rightarrow$

$|x_1-1|+|x_2-1| \underset{>D_2}{\overset{D_1}{<}} |x_1+1|+|x_2+1|$

1: $x_1<-1$, $x_2<-1$

$-x_1+1-x_2+1 \sim -x_1-1-x_2-1 \Rightarrow 2\not<-2 \Rightarrow D_2$

$x_1<-1$, $1>x_2>-1$

$-x_1+1-x_2+1 \sim -x_1-1+x_2+1 \Rightarrow x_2\not>2 \Rightarrow D_2$ (For D_1)
$2\geq x_2$

$1>x_1>-1$, $x_2<-1$

$-x_1+1-x_2+1 \sim x_1+1-x_2-1 \Rightarrow 2\geq x_1 \Rightarrow D_2$

$1>x_1>-1$, $1>x_2>-1$

$-x_1+1-x_2+1 \sim x_1+1+x_2+1$ $\quad -x_1<x_2 \Rightarrow D_1$
$\quad -x_1>x_2 \Rightarrow D_2$

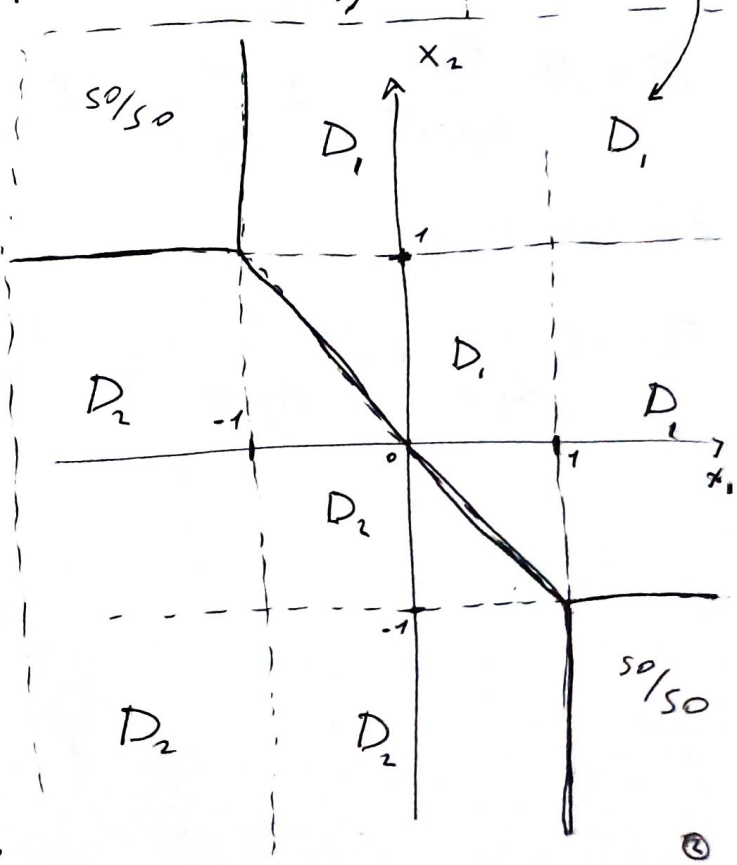
$x_1<-1$, $x_2>1$ and $x_2<-1$, $x_1>1$

$-x_1+1+x_2-1 \_\_ -x_1-1+x_2+1$ $\quad x_1-1-x_2+1 \_\_ x_1+1-x_2-1$

$0\sim 0$ $\quad 0\sim 0$

$\Downarrow$ never mind $\quad \Downarrow$ never mind 50/50

For these fields, we determine the values symmetrically, like for D_2



(3) $B_{minimal\ band} = \frac{R_s}{2} \leftarrow PSK, QAM$

$B_{minimal\ band} = M \cdot R_s \leftarrow FSK$, ortogonal

; $R_b = R$

; $\eta = \frac{R_b}{B}$ spectral efficiency

1) 64-QAM

$B = \frac{R_b}{2 \cdot \log_2 64} = \frac{R}{2 \cdot 6} = \frac{R}{12}$; $\eta = \frac{R_b}{B} \Rightarrow \eta = 12$

2) 8-PSK:

$B = \frac{R_b}{2 \cdot \log_2 8} = \frac{R}{6}$; $\eta \approx 6$

3) QPSK = 4-PSK

$B = \frac{R_b}{2 \cdot \log_2 4} = \frac{R}{4}$; $\eta \approx 4$

4) BPSK = 2-PSK

$B = \frac{R_b}{2 \cdot \log_2 2} = \frac{R}{4}$; $\eta \approx 2$

5) BFSK = 2-FSK

$B = M \cdot \frac{R_b}{\log_2 M} = 2 \cdot \frac{R}{\log_2 2} = 2R$; $\eta \approx 0,5$

6) 16-FSK

$B = M \cdot \frac{R_b}{\log_2 M} = 16 \cdot \frac{R}{\log_2 16} = \frac{16}{4} \cdot R = R \cdot 4$; $\eta \approx 0.25$.

(4) $s(t) = \begin{cases} \frac{A}{T} t \cdot \cos(2\pi f_c t), & 0 \le t \le T \\ 0, & \text{otherwise} \end{cases}$

1) $h(t) = K \cdot s(T-t)$, $K=1$ ← w/o G

$h(t) = \begin{cases} \frac{A}{T}(T-t)\cos(2\pi f_c (T-t)), & 0 \le t \le T \\ 0, & \text{otherwise} \end{cases}$

---

2) $y(t) = \int_{-\infty}^{+\infty} s(\tau) \cdot h(t-\tau)\, d\tau = s * h(t)$, $y(T) - ?$

$y(T) = \int_{-\infty}^{+\infty} s(\tau) \cdot s(T-(T-\tau))\, d\tau = \int_0^T s^2(\tau)\, d\tau = \frac{A^2}{T^2} \int_0^T \tau^2 \cdot \cos^2(2\pi f_c \tau)\, d\tau =$

$= \frac{A^2}{T^2} \left[\frac{(6T^2(2\pi f_c)^2 - 3)\cdot \sin(2T \cdot 2\pi f_c) + 6T\, 2\pi f_c \cdot \cos(2T \cdot 2\pi f_c) + 4T^3 (2\pi f_c)^3}{24 (2\pi f_c)^3} \right] =$

$= \frac{A^2 T}{6} + \frac{1}{4} \cdot \frac{A^2}{T} \cdot \frac{1}{(2\pi f_c)^2}$, $f_c \simeq 1\,GHz = 10^9 \Rightarrow y(T) \approx \frac{A^2 T}{6}$

$\frac{A^2}{16\pi^2 T} \cdot \frac{1}{f_c^2} = 0$

---

3) $R_{ss}(t) = \int_{-\infty}^{+\infty} s(\tau)\, s(\tau - t)\, d\tau$; $R_{ss}(T) = \int_0^T s^2(\tau)\, d\tau$

$R_{ss}(T) = y(T) = \frac{A^2 T}{6} = E_s = \int_{-\infty}^{+\infty} s^2(t)\, dt$

The matched filter and the correlator are equivalent, and their maximum response equals the signal energy.

main

```python
import numpy as np
import matplotlib.pyplot as plt
import random

from scipy.stats import norm

from Converter_dB import convert_SNR_dB_to_SNR, convert_SNR_to_SNR_dB
from AWGN_generator import add_AWGN_into_sequence
from Plotting_constelation import plot_constellation
from BPSK_mapper import BPSK_mapper
from BPSK_receiver import BPSK_receiver


PI = np.pi

## task addendum, not official
# An error may occur here due to the way the function=demodulate_symbols_in_time  operates.
# For large values of bit_num, an error accumulates when the signal
# is divided into symbol periods and assigned to specific symbols.
# As a result, part of the bits is lost during demodulation.
# To avoid this, increase the bit_rate, or reduce the subdivision of
# the time variable t, but not excessively, since the function may
# incorrectly compute the intervals. Alternatively, to avoid this issue,
# use a fixed number of samples by rewriting the demodulation function.
##

## Model parameters
Energy_of_impulse = 1


bit_rate = 1 # need correct setting
numb_bits = int(40000)



# snrs_dB
SNRs_dB = np.arange(10, 11, 0.25)

# result BERs
BERs = np.zeros(np.size(SNRs_dB))
BERs_teor= np.zeros(np.size(SNRs_dB))


## Create objects
bpsk_mapper = BPSK_mapper(bit_rate, Energy_of_impulse)
bpsk_demod = BPSK_receiver(bit_rate, Energy_of_impulse, np.array([0.5, 0.5]))


```

```
49  ## Start performind model of receiving
50  for num_ex in range(0, np.size(SNRs_dB)):
51      cur_SNR_dB = SNRs_dB[num_ex]
52
53      # Generate bit sequene
54      bit_seq_tx = [random.randint(0, 1) for _ in range(numb_bits)]
55
56      # Set correct time division
57      T_symb = bpsk_mapper._T_symb
58      dt = T_symb/3 + 1e-6
59      t_start = 0
60      t_end = t_start + np.size(bit_seq_tx)/bit_rate - dt
61      t = np.arange(t_start, t_end, dt)
62
63      # Mapping
64      baseband_signal = bpsk_mapper.get_baseband_signal(bit_seq_tx)
65      baseband_signals_out = baseband_signal(t)
66
67      # add AWGN
68      baseband_signal_with_noise = add_AWGN_into_sequence((baseband_signals_ou
t), cur_SNR_dB)
69
70      # demodulate
71      bit_seq_rx = bpsk_demod.demodulate_symbols_in_time(t, baseband_signal_wit
h_noise, cur_SNR_dB)
72
73      # pltting constelation
74      if cur_SNR_dB == 10:
75          plot_constellation(baseband_signals_out, baseband_signal_with_noise,
"Constelation for SNR_dB=" + str(cur_SNR_dB))
76
77      # solving BER
78      num_errors = np.sum(bit_seq_tx != bit_seq_rx) # there is may be an error
79      BERs[num_ex] = num_errors / np.size(bit_seq_tx)
80
81      # Theoretical value of error's probability
82      BERs_teor[num_ex] = norm.sf(np.sqrt(2*convert_SNR_dB_to_SNR(cur_SNR_dB)))
83
84      print(cur_SNR_dB)
85
86
87
88  ## Plotting results BER curves
89  fig5, axs = plt.subplots(1, 1, figsize=(10, 8))
90
91  axs.semilogy(SNRs_dB, BERs, '.', label='Practical error\'s probability')
92  axs.semilogy(SNRs_dB, BERs_teor , '--', label='Teoretical error\'s probabilit
y')
93  axs.set_title('curves of BER')
94  axs.legend()
95  axs.set_xlabel('SNR dB')
96  axs.set_ylabel('BER')
```

```
axs.grid(True)

plt.tight_layout()
plt.show()
```

BPSK_receiver

```python
import numpy as np

from Converter_dB import convert_SNR_dB_to_SNR, convert_SNR_to_SNR_dB


PI = np.pi

def int_to_bits_lsb(value: int, length: int) -> np.ndarray:

    if value < 0:
        raise ValueError("value >= 0 must be")
    if value >= (1 << length):
        raise ValueError("value size not combinian into import length")

    return np.array([(value >> i) & 1 for i in range(length)],
                    dtype=np.uint8)



class BPSK_receiver:

    def __init__(self, bit_rate, avg_signal_energy, prob_m_arr):

        self._bit_rate              = bit_rate
        self._T_symb                = 1/self._bit_rate
        self._E_symb_avg            = avg_signal_energy
        self._mod_symbols           = self._get_mod_arr()
        self._demod_symbols         = self._get_demod_dict()
        self._prob_m                = prob_m_arr



    def _get_mod_arr(self):
        arr_signals = np.zeros(2, dtype=complex)

        for numb in range(0, 2):
            bits = int_to_bits_lsb(numb, 1)
            arr_signals[numb] = (1/np.sqrt(2))*((1 - 2*bits[0]) + 1j*(1 - 2*b
its[0]))

        return arr_signals * np.sqrt(self._E_symb_avg)


    def _get_demod_dict(self):
        dict_simbols = {}

        for num in range(0, 2):
            dict_simbols[self._mod_symbols[num]] = int_to_bits_lsb(num, 1)

```

```
49            return dict_simbols
50
51
52        def _demodulate_signal(self, t, symb_seq, snr_db):
53            # by distanse metric, MAP algorithm
54
55            if (np.size(t) - np.size(symb_seq) != 1): raise ValueError("not corre
ctlength in input params, t must be greater than symb_seq by 1")
56
57            snr_lin = convert_SNR_dB_to_SNR(snr_db)
58            N_0 = self._E_symb_avg/snr_lin
59
60            dist_m = np.zeros(np.size(self._mod_symbols))
61
62            for m in range(0, np.size(self._mod_symbols)):
63                signal_m = self._mod_symbols[m]
64                cur_dist_m = 0
65
66                for i in range(0, (np.size(t) - 1)):
67                    cur_dist_m += np.power(np.abs(symb_seq[i] - signal_m), 2) *
(t[i+1]-t[i])
68
69                dist_m[m] = N_0 * np.log(self._prob_m[m]) - cur_dist_m
70
71            return self._demod_symbols[self._mod_symbols[np.argmax(dist_m)]]
72
73
74        def demodulate_symbols_in_time(self, t, signals_in, snr_db):
75
76            if not(np.size(t) >= 2):  raise ValueError("t size must be >= 2")
77            if not(np.all(np.diff(t) >= 0)):  raise ValueError("t[i] >= t[i+1]")
78            if np.size(t) != np.size(signals_in): raise ValueError("not same leng
th t and symbols_in")
79
80            demod_seq = []
81
82            cur_num_t_start = 0
83
84            for i in range(1, np.size(t)):
85
86                if (t[i] - t[cur_num_t_start] >= self._T_symb):
87                    cur_bit_seq = self._demodulate_signal(t[cur_num_t_start:(i+
1)], signals_in[cur_num_t_start:i], snr_db)
88                    cur_num_t_start = i
89                    demod_seq.append(cur_bit_seq)
90
91                elif (t[i] == t[-1]) and (cur_num_t_start != i):
92                    cur_bit_seq = self._demodulate_signal(t[cur_num_t_start:(i+
1)], signals_in[cur_num_t_start:i], snr_db)
93                    demod_seq.append(cur_bit_seq)
94
95                else:
```

```
                continue

        return np.concatenate(demod_seq)
```

BPSK_mapper

```python
import numpy as np


def int_to_bits_lsb(value: int, length: int) -> np.ndarray:

    if value < 0:
        raise ValueError("value >= 0 must be")
    if value >= (1 << length):
        raise ValueError("value size not combinian into import length")

    return np.array([(value >> i) & 1 for i in range(length)],
                    dtype=np.uint8)


def bits_to_int_lsb(bits):
    value = 0

    for i, bit in enumerate(bits):
        value |= int(bit) << i

    return value



class BPSK_mapper:

    def __init__(self, bit_rate, E_symb_avg=1):

        self._bit_rate              = bit_rate
        self._num_bits_in_qam       = 1
        self._symbol_rate           = self._bit_rate/self._num_bits_in_qam
        self._T_symb                = 1/self._symbol_rate
        self._E_symb_avg            = E_symb_avg
        self._E_bit_avg             = E_symb_avg
        self._mod_signals           = self._get_mod_arr()



    def _get_mod_arr(self):
        arr_signals = np.zeros(2, dtype=complex)

        for numb in range(0, 2):
            bits = int_to_bits_lsb(numb, 1)
            arr_signals[numb] = (1/np.sqrt(2))*((1 - 2*bits[0]) + 1j*(1 - 2*b
its[0]))

        return arr_signals * np.sqrt(self._E_symb_avg)


```

```
49      def get_average_energy(self):
50
51          return sum(x * x for x in self._mod_signals) / 2
52
53
54      def get_baseband_signal(self, bit_sequence):
55
56          if ((np.size(bit_sequence) % self._num_bits_in_qam) != 0): raise Valu
eError("the number of bits is not a multiple of the number of bits per symbo")
57
58          bit_seq_sz = np.size(bit_sequence)
59
60          def baseband_signal_t(t):
61
62              n = t // self._T_symb
63              start_bits = int(n*self._num_bits_in_qam)
64              end_bits = int((n+1)*self._num_bits_in_qam)
65
66              if start_bits >= bit_seq_sz or end_bits > bit_seq_sz: raise Value
Error("There aren't bits for do modulating")
67
68              cur_num = bits_to_int_lsb(bit_sequence[start_bits:end_bits])
69
70              return self._mod_signals[cur_num]
71
72          def baseband_signal(t_arr):
73              if np.isscalar(t_arr):
74                  return baseband_signal_t(t_arr)
75
76              result = np.zeros_like(t_arr, dtype=complex)
77
78              for i in range(0, np.size(t_arr)):
79                  result[i] = baseband_signal_t(t_arr[i])
80
81              return result
82
83          return baseband_signal
84
85
86      def get_modulate_sequence_and_time(self, t_start, t_end, numb_of_points,
bit_sequence):
87
88          if (t_end - t_start) <= 0:  raise ValueError("t_end <= t_start")
89          if ((t_end - t_start) // self._T_symb + 1) * self._num_bits_in_qam >
np.size(bit_sequence): raise ValueError("Unsufficient bits to form the baseband sign
al")
90          if numb_of_points <= 0: raise ValueError("numb_points must be > 0")
91
92          t = np.linspace(t_start, t_end, numb_of_points)
93          result = np.zeros(numb_of_points, dtype=complex)
94
95          for i in range(0, numb_of_points):
```

```
            n = (t[i] - t[0]) // self._T_symb

            if n >= np.size(numb_of_points):
                result[i] = 0
                continue

            start_bits = int(n*self._num_bits_in_qam)
            end_bits = int((n+1)*self._num_bits_in_qam)

            cur_signal_val = bits_to_int_lsb(bit_sequence[start_bits:end_bit
s])


            result[i] = self._mod_signals[cur_signal_val]

        return result, t
```

AWGN_generator

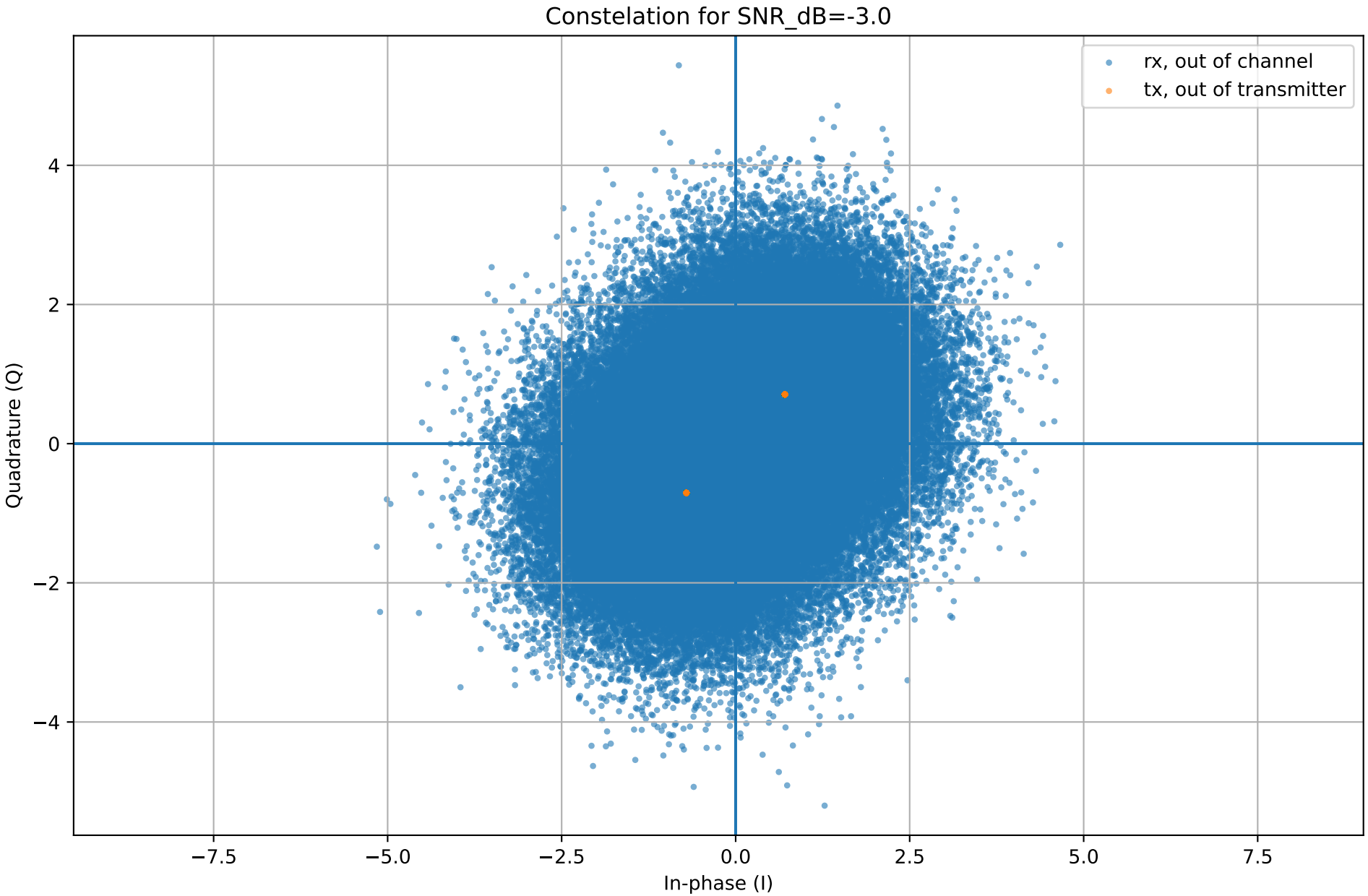
```python
import numpy as np
import Converter_dB as conv_dB


def add_AWGN_into_sequence(signal_seq, snr_db):

    power_signal = np.mean(np.power(np.abs(signal_seq), 2))
    snr_linear = conv_dB.convert_SNR_dB_to_SNR(snr_db)
    power_noise = power_signal / snr_linear

    if np.iscomplexobj(signal_seq):
        noise = np.sqrt(power_noise/2) * (np.random.randn(*signal_seq.shape)
+ 1j*np.random.randn(*signal_seq.shape))
    else:
        noise = np.sqrt(power_noise) * np.random.randn(*signal_seq.shape)

    return signal_seq + noise

```

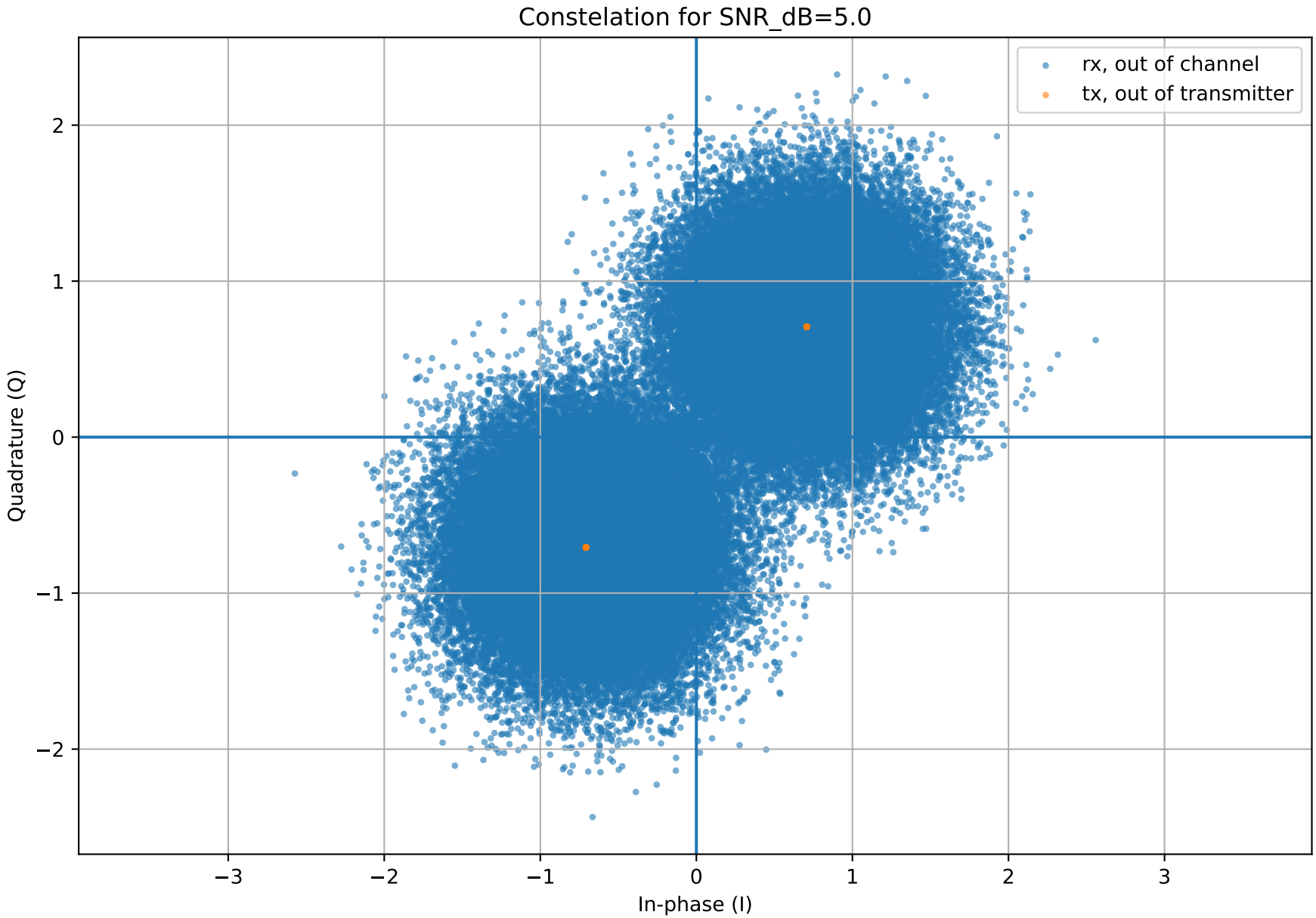
Converter_dB

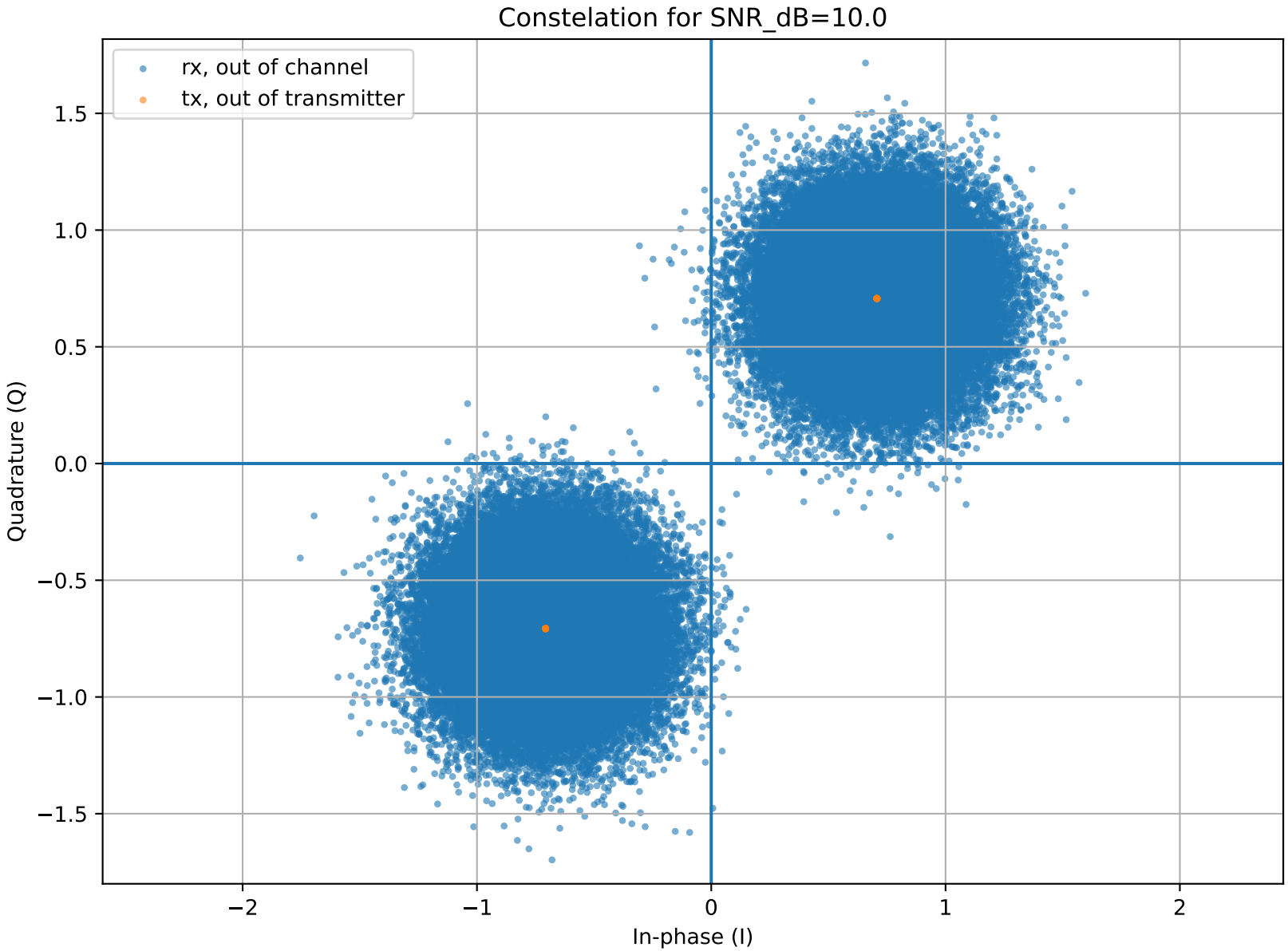
```python
import numpy as np


def convert_SNR_to_SNR_dB(SNR):

    return 10*np.log10(SNR)


def convert_SNR_dB_to_SNR(SNR_dB):

    return np.power(10, SNR_dB/10)
```

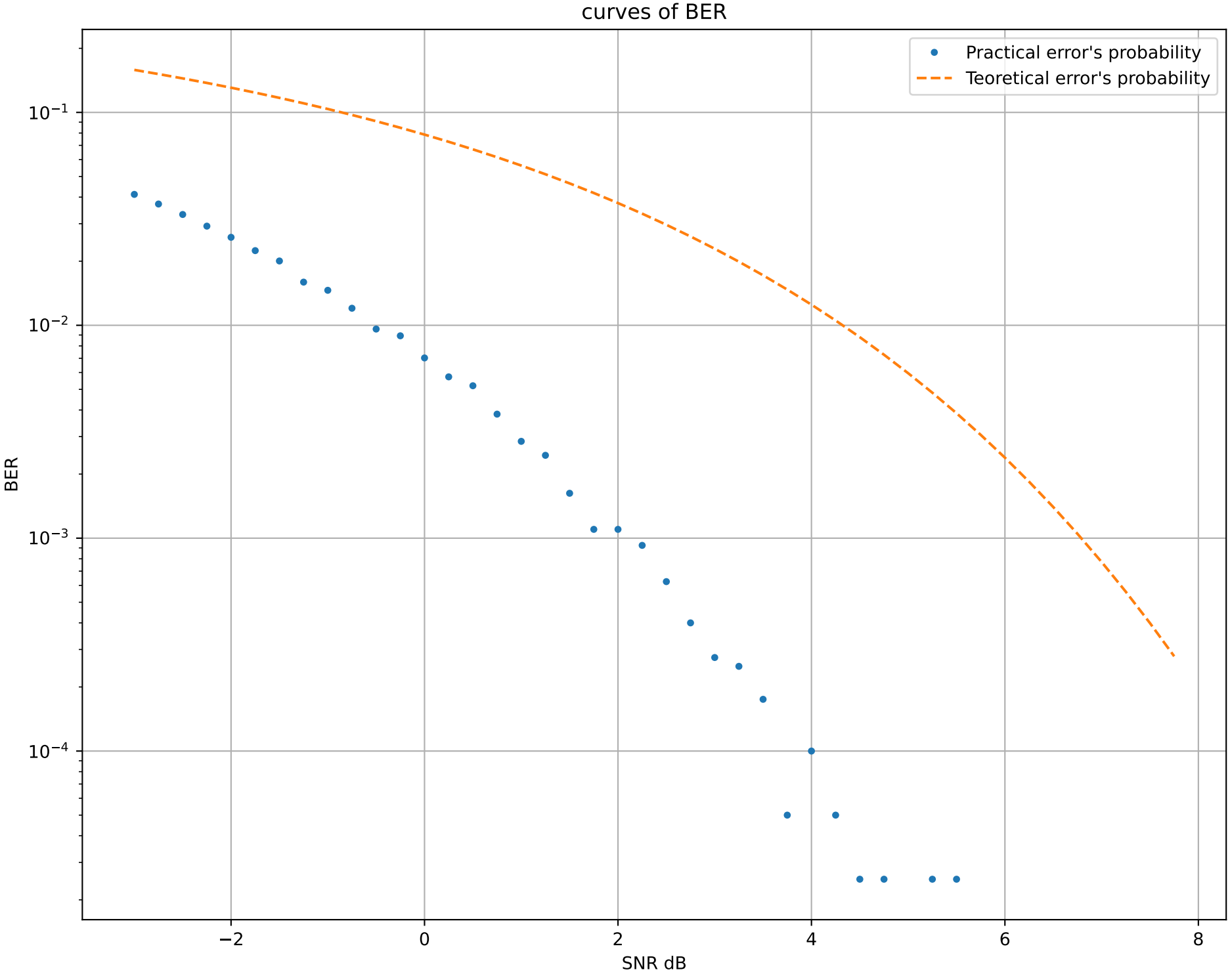
Plotting_constellation

```
import numpy as np
import matplotlib.pyplot as plt

def plot_constellation(signals_tx, signals_rx, title="Constellation diagra
m"):
    plt.figure()
    plt.scatter(signals_rx.real, signals_rx.imag, s=5, alpha=0.6, label='rx,
out of channel')
    plt.scatter(signals_tx.real, signals_tx.imag, s=5, alpha=0.6, label='tx,
out of transmitter')
    plt.axhline(0)
    plt.axvline(0)
    plt.xlabel("In-phase (I)")
    plt.ylabel("Quadrature (Q)")
    plt.title(title)
    plt.grid(True)
    plt.axis('equal')
    plt.legend()
    plt.show()

```


Constelation for SNR_dB=-3.0
rx, out of channel
tx, out of transmitter
Quadrature (Q)
In-phase (I)
4
2
0
−2
−4
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5


Constelation for SNR_dB=5.0
rx, out of channel
tx, out of transmitter
Quadrature (Q)
In-phase (I)
2
1
0
−1
−2
−3
−2
−1
0
1
2
3


Constelation for SNR_dB=10.0
rx, out of channel
tx, out of transmitter
Quadrature (Q)
In-phase (I)
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2
−1
0
1
2


curves of BER
Practical error's probability
Teoretical error's probability
BER
SNR dB
10^{-1}
10^{-2}
10^{-3}
10^{-4}
−2
0
2
4
6
8